House Price Prediction

Introduction:

This project titled as “House Price Prediction”, developed to predict house prices based on real world data considering features like number of bedrooms, bathrooms, built year, area in sqft, and number of floors, etc, using a machine learning model. Trained the model using seven different types of Machine Learning regression algorithms. Before training the model, I applied feature scaling using StandardScalar to normalize all features equally by preventing features with high magnitude dominating the features with low magnitude. Then evaluated the model after training using different regression metrics.

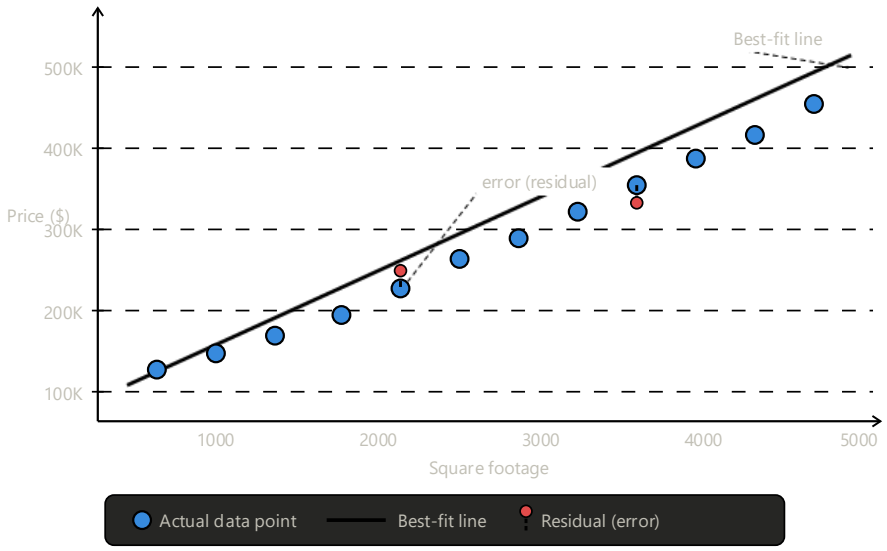
Dataset used: house_data.csv

The dataset was divided into training data and testing data. To train and evaluate the model.

Machine Learning Algorithms used:

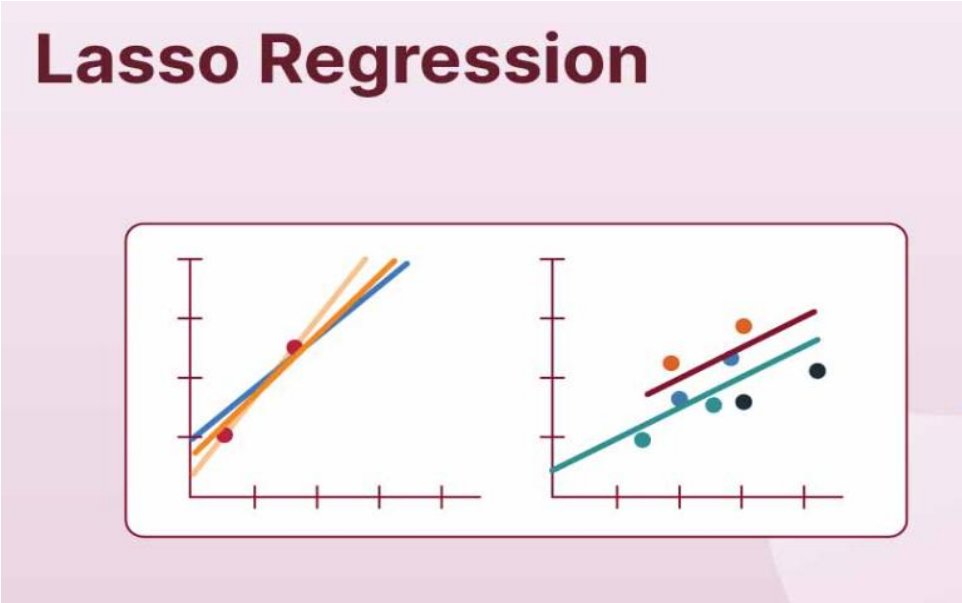
1. Linear Regression:

Linear Regression is simplest supervised learning algorithms used for predicting continuous values which are numbers like house price. It works by finding the best-fit straight line through the data points that minimizes the difference between the actual and predicted values. The algorithm learns the relationship between the independent variables (features) and the dependent variable (target) by estimating the coefficients (weights) for each feature. It is fast and easy to interpret but may not be accurate when the data is non-linear.



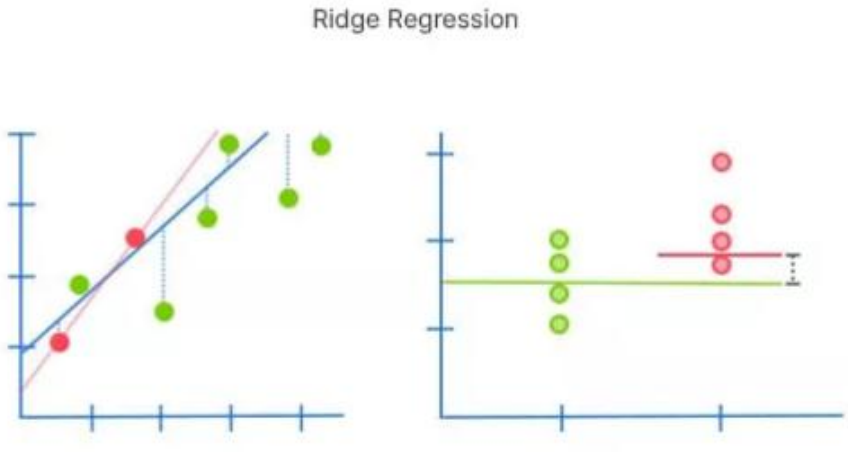
2. Lasso Regression:

Lasso Regression is just like a linear regression that adds L1 regularization to improve model generalization by adding a penalty. This penalty not only controls overfitting but also performs feature selection by shrinking some coefficients to completely zero, which means it automatically removes the features it considers less important from the model. It is particularly useful when the dataset has many features and only a few of them are contributing to the prediction.



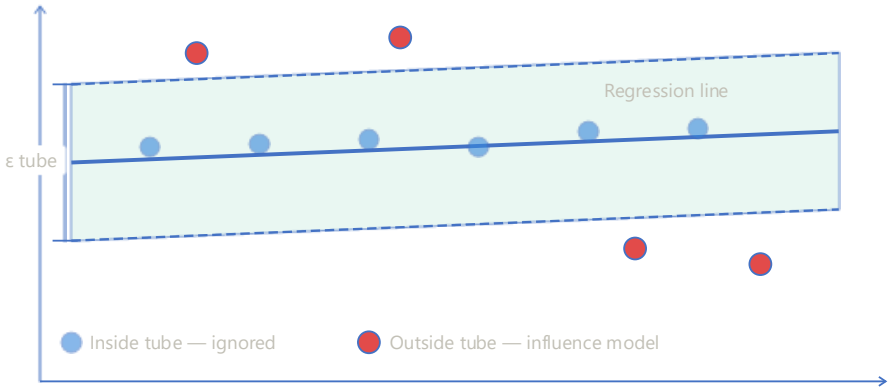
3. Ridge Regression:

Ridge Regression is an improved version of Linear Regression that adds regularization to avoid overfitting problem in the data. It works exactly like Linear Regression but adds an additional penalty term called L2 regularization to the cost function, it keeps all the features balanced, so no single feature gets to dominate the others in the decision-making process. Ridge Regression is particularly useful when the dataset has many correlated features.



4. Support Vector Regressor:

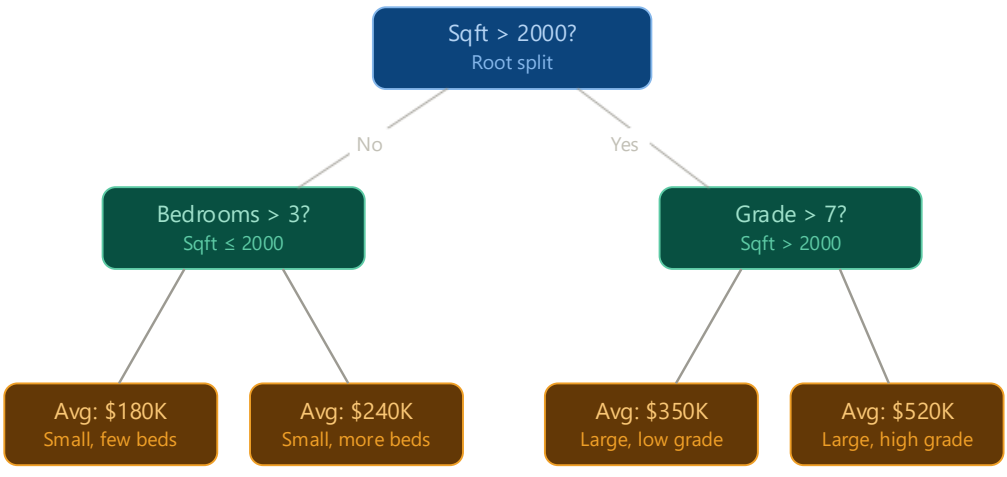
Support Vector Regressor(SVR) is the regression version of the Support Vector Machine algorithm, used for predicting continuous values. It works by finding the best-fit line within a defined margin of tolerance called the epsilon tube. It tries to keep the predictions as simple and stable as possible so that it performs well not just on the training data but also on new and unseen data. It handles complex relationships well but is sensitive to parameter tuning and requires scaled data.



Support Vector Regressor — Epsilon tube keeps predictions stable

5. Decision tree Regressor:

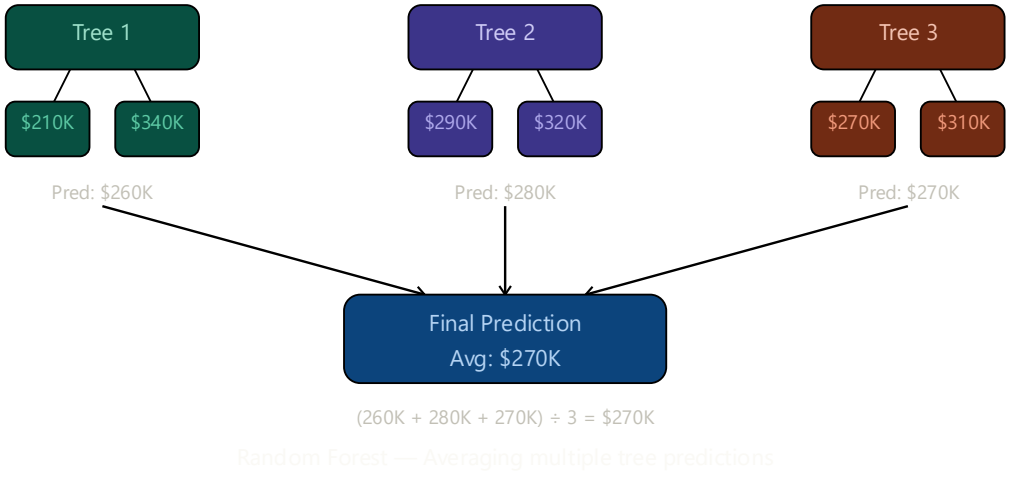
Decision Tree Regressor works the same way as a classification decision tree, but it predicts a continuous numerical value instead of a categorical value. It splits the data repeatedly based on feature conditions, trying to group similar values together at every step until it reaches a final prediction. The predicted value at each leaf is typically the average of all the data points that fall into that region. It handles non-linear data well but can overfit if the tree grows too deep.



Decision Tree Regressor — Splits leading to average predictions

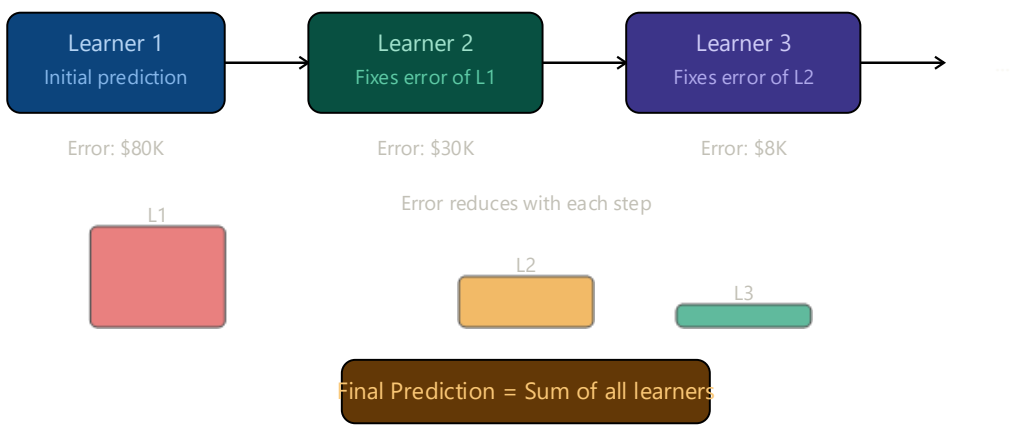
6. Random Forest Regressor:

Random forest is an ensemble learning method that uses multiple decision trees for prediction. Each tree is trained on a random subset of data and features. The final output is predicted by averaging the outputs of all single decision trees. This reduces overfitting and gives more stable, accurate predictions compared to a single tree. It also provides feature importance scores, making it useful for understanding which variables are most important for predictions.



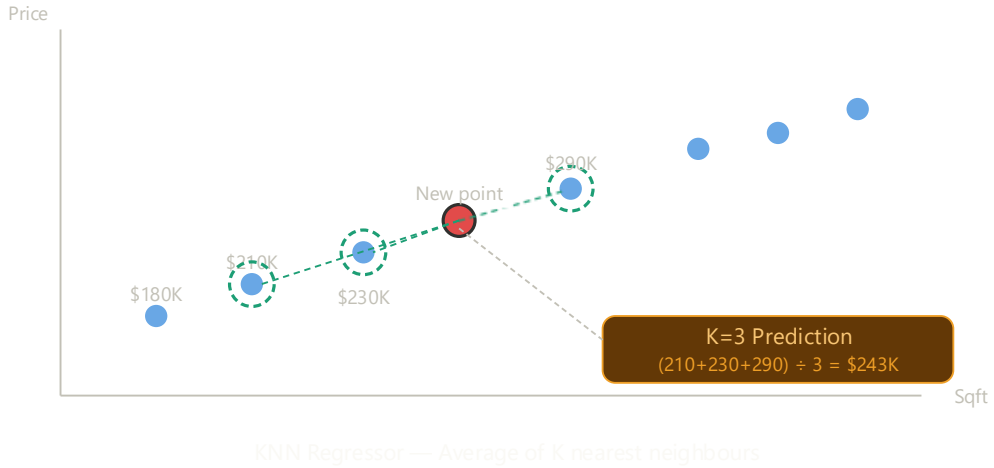
7. Gradient Boosting Regressor:

Gradient boosting regressor is a powerful ensemble learning algorithm, that builds weak learners usually decision tress, sequentially where each new learner focuses on correcting the errors made by the previous one. Unlike Random Forest which builds models independently and in parallel, Gradient Boosting learns step by step, gradually improving its predictions with every iteration. It is one of the most accurate models but highly sensitive to noisy data if not tuned properly.



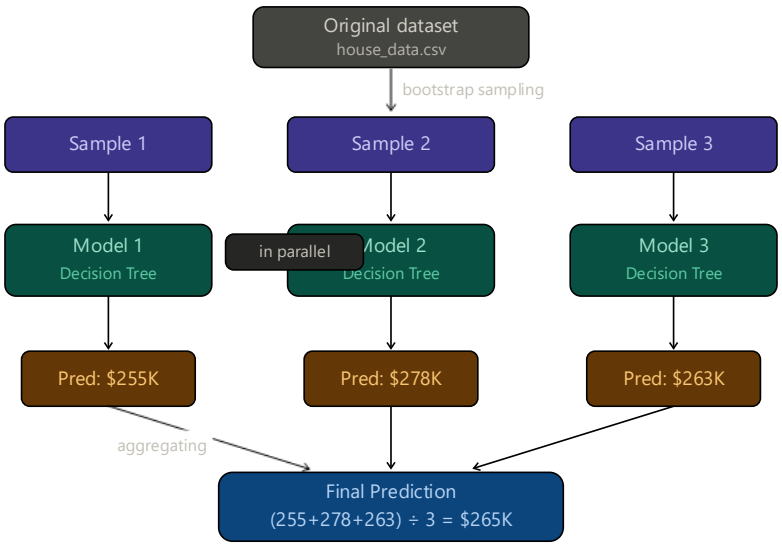
8. K-Nearest Neighbour Regressor:

K-Nearest Neighbour (KNN) regressor is a simple and intuitive algorithm that predicts the value of a new data point by looking at the K closest data points in the training set and taking their average as the final prediction. It does not build a model explicitly during training, instead it memorizes the entire dataset and makes predictions, which is why it is called a lazy learner. It works well when the data is scaled and the features are meaningful and relevant but slows down with large datasets and needs feature scaling.



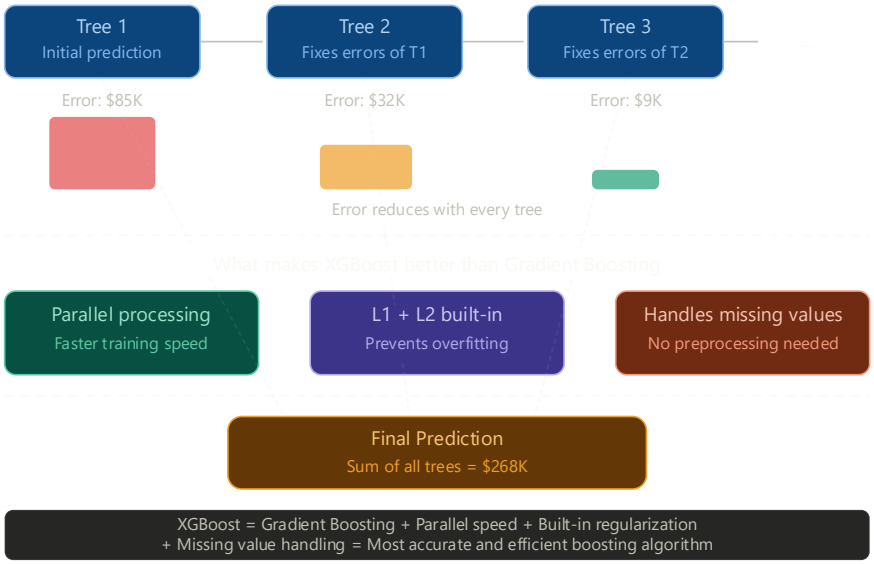
9. Bagging Regressor:

Bagging Regressor is an ensemble learning method that stands for Bootstrap Aggregating. It combines multiple base regression models (typically decision trees) to improve model performance and reduce variance. It works by creating multiple random subsets of the training data through a process called bootstrap sampling, where data is sampled with replacement. Model is trained on each of these subsets independently and in parallel. The final prediction is made by averaging the outputs of all individual models.



10.XGBoost Regressor:

XGBoost stands for Extreme Gradient Boosting, it is an upgraded version of Gradient Boosting that builds weak learners sequentially, where each new learner focuses on correcting the errors of the previous one. XGBoost is significantly faster than regular Gradient Boosting because it supports parallel processing. It also has built-in L1 and L2 regularization which automatically prevents overfitting and it can also handle missing values on its own without requiring any extra preprocessing. These combined improvements make XGBoost one of the most accurate and efficient algorithm.



Conclusion:

This project is based on predicting house prices and since the output is a continuous numerical value, regression based machine learning algorithms were the best choice here. Model is trained using different algorithms including Linear Regression, Lasso Regression, Ridge Regression, Support Vector Regressor, Decision Tree, Random Forest, Gradient Boosting, K-Nearest Neighbour, Bagging Regressor, and XGBooster Regressor. All trained on the same dataset and evaluated using performance metrics like R2 Score, MAE, and RMSE. The results revealed that ensemble methods like Random Forest and Gradient Boosting delivered stronger and more reliable accuracy compared to simpler models. Linear models held up well where the data had a clear linear pattern, while tree based models proved more effective in capturing complex and non-linear relationships within the data. Ultimately, the best model can always be chosen based on how well it performs and what the specific application demands.

What I have Learned

One of the best thing I learned from this project is that there is no single algorithm that fits every problem each one has its own strengths and limitations depending on the data and the task at hand. Working through seven different algorithms on the same dataset gave me a much clear understanding of how differently each model thinks and behaves. I also understood that choosing the wrong algorithm can lead to underfitting or overfitting, both of which effect the model's ability to generalize on new data. It taught me to be more thoughtful and deliberate when selecting a model rather than just going with the first one that comes to mind. Finally, evaluation metrics like R2 Score, MAE, and RMSE turned out to be incredibly valuable tools they take all the complexity of a model's performance and bring it down to a single, easy to interpret number that tells you exactly how well your model is doing.